

org.apache.spark.sql

Experimental

Class DataFrame

java.lang.Object
org.apache.spark.sql.DataFrame

All Implemented Interfaces:
java.io.Serializable, org.apache.spark.sql.execution.Queryable

```
public class DataFrame  
extends java.lang.Object  
implements org.apache.spark.sql.execution.Queryable, scala.Serializable
```

A distributed collection of data organized into named columns.
A `DataFrame` is equivalent to a relational table in Spark SQL. The following example creates a `DataFrame` by pointing Spark SQL to a Parquet data set.

```
val people = sqlContext.read.parquet("...") // in Scala  
DataFrame people = sqlContext.read().parquet("...") // in Java
```

Once created, it can be manipulated using the various domain-specific-language (DSL) functions defined in: `DataFrame` (this class), `Column`, and `functions`.

To select a column from the data frame, use `apply` method in Scala and `col` in Java.

```
val ageCol = people("age") // in Scala  
Column ageCol = people.col("age") // in Java
```

Note that the `Column` type can also be manipulated through its various functions.

```
// The following creates a new column that increases everybody's age by 10.  
people("age") + 10 // in Scala  
people.col("age").plus(10); // in Java
```

A more concrete example in Scala:

```
// To create DataFrame using SQLContext  
val people = sqlContext.read.parquet("...")  
val department = sqlContext.read.parquet("...")  
  
people.filter("age > 30")  
  .join(department, people("deptId") === department("id"))  
  .groupBy(department("name"), "gender")  
  .agg(avg(people("salary")), max(people("age")))
```

and in Java:

```
// To create DataFrame using SQLContext  
DataFrame people = sqlContext.read().parquet("...");  
DataFrame department = sqlContext.read().parquet("...");  
  
people.filter("age".gt(30))  
  .join(department, people.col("deptId").equalTo(department("id")))  
  .groupBy(department.col("name"), "gender")  
  .agg(avg(people.col("salary")), max(people.col("age")));
```

Since:

1.3.0

See Also:

[Serialized Form](#)

Constructor Summary

Constructors

Constructor and Description

<code>DataFrame(SQLContext sqlContext, org.apache.spark.sql.catalyst.plans.logical.LogicalPlan logicalPlan)</code> A constructor that automatically analyzes the logical plan.

Method Summary

Methods	
Modifier and Type	Method and Description

<code>DataFrame</code>	<code>agg(Column expr, Column... exprs)</code> Aggregates on the entire <code>DataFrame</code> without groups.
<code>DataFrame</code>	<code>agg(Column expr, scala.collection.Seq<Column> exprs)</code> Aggregates on the entire <code>DataFrame</code> without groups.
<code>DataFrame</code>	<code>agg(scala.collection.immutable.Map<java.lang.String, java.lang.String> expr)</code> (Scala-specific) Aggregates on the entire <code>DataFrame</code> without groups.
<code>DataFrame</code>	<code>agg(java.util.Map<java.lang.String, java.lang.String> exprs)</code> (Java-specific) Aggregates on the entire <code>DataFrame</code> without groups.
<code>DataFrame</code>	<code>agg(scala.Tuple2<java.lang.String, java.lang.String> aggExpr, scala.collection.Seq<scala.Tuple2<java.lang.String, java.lang.String>> aggs)</code> (Scala-specific) Aggregates on the entire <code>DataFrame</code> without groups.
<code>DataFrame</code>	<code>alias(java.lang.String alias)</code> Returns a new <code>DataFrame</code> with an alias set.
<code>DataFrame</code>	<code>alias(scala.Symbol alias)</code> (Scala-specific) Returns a new <code>DataFrame</code> with an alias set.
<code>Column</code>	<code>apply(java.lang.String colName)</code> Selects column based on the column name and return it as a <code>Column</code> .
<code><U> Dataset<U></code>	<code>as(Encoder<U> evidence\$1)</code> Converts this <code>DataFrame</code> to a strongly-typed <code>Dataset</code> containing objects of the specified type <code>U</code> .
<code>DataFrame</code>	<code>as(java.lang.String alias)</code> Returns a new <code>DataFrame</code> with an alias set.
<code>DataFrame</code>	<code>as(scala.Symbol alias)</code> (Scala-specific) Returns a new <code>DataFrame</code> with an alias set.
<code>DataFrame</code>	<code>cache()</code> Persist this <code>DataFrame</code> with the default storage level (<code>MEMORY_AND_DISK</code>).
<code>DataFrame</code>	<code>coalesce(int numPartitions)</code> Returns a new <code>DataFrame</code> that has exactly <code>numPartitions</code> partitions.
<code>Column</code>	<code>col(java.lang.String colName)</code> Selects column based on the column name and return it as a <code>Column</code> .
<code>Row[]</code>	<code>collect()</code> Returns an array that contains all of <code>Rows</code> in this <code>DataFrame</code> .
<code>java.util.List<Row></code>	<code>collectAsList()</code> Returns a Java list that contains all of <code>Rows</code> in this <code>DataFrame</code> .
<code>protected int</code>	<code>collectToPython()</code>
<code>java.lang.String[]</code>	<code>columns()</code> Returns all column names as an array.
<code>long</code>	<code>count()</code> Returns the number of rows in the <code>DataFrame</code> .
<code>void</code>	<code>createJDBCTable(java.lang.String url, java.lang.String table, boolean allowOverwrite)</code> Deprecated. <i>As of 1.3.40, replaced by <code>write().jdbc()</code>. This will be removed in Spark 2.0.</i>
<code>GroupedData</code>	<code>cube(Column... cols)</code> Create a multi-dimensional cube for the current <code>DataFrame</code> using the specified columns, and aggregation on them.
<code>GroupedData</code>	<code>cube(scala.collection.Seq<Column> cols)</code> Create a multi-dimensional cube for the current <code>DataFrame</code> using the specified columns, and aggregation on them.
<code>GroupedData</code>	<code>cube(java.lang.String col1, scala.collection.Seq<java.lang.String> cols)</code> Create a multi-dimensional cube for the current <code>DataFrame</code> using the specified columns, and aggregation on them.
<code>GroupedData</code>	<code>cube(java.lang.String col1, java.lang.String... cols)</code> Create a multi-dimensional cube for the current <code>DataFrame</code> using the specified columns, and aggregation on them.
<code>DataFrame</code>	<code>describe(scala.collection.Seq<java.lang.String> cols)</code> Computes statistics for numeric columns, including count, mean, stddev, min, and max.
<code>DataFrame</code>	<code>describe(java.lang.String... cols)</code> Computes statistics for numeric columns, including count, mean, stddev, min, and max.
<code>DataFrame</code>	<code>distinct()</code> Returns a new <code>DataFrame</code> that contains only the unique rows from this <code>DataFrame</code> .
<code>DataFrame</code>	<code>drop(Column col)</code> Returns a new <code>DataFrame</code> with a column dropped.
<code>DataFrame</code>	<code>drop(java.lang.String colName)</code> Returns a new <code>DataFrame</code> with a column dropped.
<code>DataFrame</code>	<code>dropDuplicates()</code> Returns a new <code>DataFrame</code> that contains only the unique rows from this <code>DataFrame</code> .
<code>DataFrame</code>	<code>dropDuplicates(scala.collection.Seq<java.lang.String> colNames)</code> (Scala-specific) Returns a new <code>DataFrame</code> with duplicate rows removed, considering only the specified columns.
<code>DataFrame</code>	<code>dropDuplicates(java.lang.String[] colNames)</code> Returns a new <code>DataFrame</code> with duplicate rows removed, considering only the subset of columns specified by <code>colNames</code> .
<code>scala.Tuple2<java.lang.String, java.lang.String>[]</code>	<code>dtypes()</code> Returns all column names and their data types as an array.
<code>DataFrame</code>	<code>except(DataFrame other)</code> Returns a new <code>DataFrame</code> containing rows in this frame but not in another frame.
<code>void</code>	<code>explain()</code> Prints the physical plan to the console for debugging purposes.
<code>void</code>	<code>explain(boolean extended)</code> Prints the plans (logical and physical) to the console for debugging purposes.
<code><A extends scala.Product></code>	<code>explode(scala.collection.Seq<Column> input, scala.Function1<Row, scala.collection.TraversableOnce<A>> f, scala.reflect.api.TypeTags.TypeTag<A> evidence\$2)</code> (Scala-specific) Returns a new <code>DataFrame</code> where each row has been expanded to zero or more rows using the function <code>f</code> .
<code>DataFrame</code>	<code>explode(java.lang.String inputColumn, java.lang.String outputColumn, scala.Function1<A, scala.collection.TraversableOnce> f, scala.reflect.api.TypeTags.TypeTag evidence\$3)</code> (Scala-specific) Returns a new <code>DataFrame</code> where a single column has been expanded to zero or more columns using the function <code>f</code> .
<code><A,B> DataFrame</code>	

<code>DataFrame</code>	the provided function. <code>filter(Column condition)</code> Filters rows using the given condition.
<code>DataFrame</code>	<code>filter(java.lang.String conditionExpr)</code> Filters rows using the given SQL expression.
<code>Row</code>	<code>first()</code> Returns the first row.
<code><R> RDD<R></code>	<code>flatMap(scala.Function1<Row,scala.collection.TraversableOnce<R>> f, scala.reflect.ClassTag<R> evidence\$5)</code> Returns a new RDD by first applying a function to all rows of this <code>DataFrame</code> , and then flat
<code>void</code>	<code>foreach(scala.Function1<Row,scala.runtime.BoxedUnit> f)</code> Applies a function f to all rows.
<code>void</code>	<code>foreachPartition(scala.Function1<scala.collection.Iterator<Row>,scala.runt</code> Applies a function f to each partition of this <code>DataFrame</code> .
<code>GroupedData</code>	<code>groupBy(Column... cols)</code> Groups the <code>DataFrame</code> using the specified columns, so we can run aggregation on them.
<code>GroupedData</code>	<code>groupBy(scala.collection.Seq<Column> cols)</code> Groups the <code>DataFrame</code> using the specified columns, so we can run aggregation on them.
<code>GroupedData</code>	<code>groupBy(java.lang.String col1, scala.collection.Seq<java.lang.String> cols)</code> Groups the <code>DataFrame</code> using the specified columns, so we can run aggregation on them.
<code>GroupedData</code>	<code>groupBy(java.lang.String col1, java.lang.String... cols)</code> Groups the <code>DataFrame</code> using the specified columns, so we can run aggregation on them.
<code>Row</code>	<code>head()</code> Returns the first row.
<code>Row[]</code>	<code>head(int n)</code> Returns the first n rows.
<code>java.lang.String[]</code>	<code>inputFiles()</code> Returns a best-effort snapshot of the files that compose this <code>DataFrame</code> .
<code>void</code>	<code>insertInto(java.lang.String tableName)</code> Deprecated. <i>As of 1.4.0, replaced by write().mode(SaveMode.Append).saveAsTable(tableName). in Spark 2.0.</i>
<code>void</code>	<code>insertInto(java.lang.String tableName, boolean overwrite)</code> Deprecated. <i>As of 1.4.0, replaced by write().mode(SaveMode.Append SaveMode.Overwrite).saveAsTable(tableName). This will be removed in Spark 2.0.</i>
<code>void</code>	<code>insertIntoJDBC(java.lang.String url, java.lang.String table, boolean overw</code> Deprecated. <i>As of 1.4.0, replaced by write().jdbc(). This will be removed in Spark 2.0.</i>
<code>DataFrame</code>	<code>intersect(DataFrame other)</code> Returns a new <code>DataFrame</code> containing rows only in both this frame and another frame.
<code>boolean</code>	<code>isLocal()</code> Returns true if the collect and take methods can be run locally (without any Spark exec
<code>JavaRDD<Row></code>	<code>javaRDD()</code> Returns the content of the <code>DataFrame</code> as a <code>JavaRDD</code> of <code>Rows</code> .
<code>protected JavaRDD<byte[]></code>	<code>javaToPython()</code> Converts a <code>JavaRDD</code> to a <code>PythonRDD</code> .
<code>DataFrame</code>	<code>join(DataFrame right)</code> Cartesian join with another <code>DataFrame</code> .
<code>DataFrame</code>	<code>join(DataFrame right, Column joinExprs)</code> Inner join with another <code>DataFrame</code> , using the given join expression.
<code>DataFrame</code>	<code>join(DataFrame right, Column joinExprs, java.lang.String joinType)</code> Join with another <code>DataFrame</code> , using the given join expression.
<code>DataFrame</code>	<code>join(DataFrame right, scala.collection.Seq<java.lang.String> usingColumns)</code> Inner equi-join with another <code>DataFrame</code> using the given columns.
<code>DataFrame</code>	<code>join(DataFrame right, scala.collection.Seq<java.lang.String> usingColumns, java.lang.String joinType)</code> Equi-join with another <code>DataFrame</code> using the given columns.
<code>DataFrame</code>	<code>join(DataFrame right, java.lang.String usingColumn)</code> Inner equi-join with another <code>DataFrame</code> using the given column.
<code>DataFrame</code>	<code>limit(int n)</code> Returns a new <code>DataFrame</code> by taking the first n rows.
<code>protected org.apache.spark.sql.catalyst.plans.logical.LogicalPlan</code>	<code>logicalPlan()</code>
<code><R> RDD<R></code>	<code>map(scala.Function1<Row,R> f, scala.reflect.ClassTag<R> evidence\$4)</code> Returns a new RDD by applying a function to all rows of this <code>DataFrame</code> .
<code><R> RDD<R></code>	<code>mapPartitions(scala.Function1<scala.collection.Iterator<Row>,scala.collect scala.reflect.ClassTag<R> evidence\$6)</code> Returns a new RDD by applying a function to each partition of this <code>DataFrame</code> .
<code>DataFrameNaFunctions</code>	<code>na()</code> Returns a <code>DataFrameNaFunctions</code> for working with missing data.
<code>protected scala.collection.Seq<org.apache.spark.sql.catalyst.expressions.Expression></code>	<code>numericColumns()</code>
<code>DataFrame</code>	<code>orderBy(Column... sortExprs)</code> Returns a new <code>DataFrame</code> sorted by the given expressions.
<code>DataFrame</code>	<code>orderBy(scala.collection.Seq<Column> sortExprs)</code> Returns a new <code>DataFrame</code> sorted by the given expressions.
<code>DataFrame</code>	<code>orderBy(java.lang.String sortCol, scala.collection.Seq<java.lang.String> s</code> Returns a new <code>DataFrame</code> sorted by the given expressions.
<code>DataFrame</code>	<code>orderBy(java.lang.String sortCol, java.lang.String... sortCols)</code> Returns a new <code>DataFrame</code> sorted by the given expressions.
<code>DataFrame</code>	<code>persist()</code> Persist this <code>DataFrame</code> with the default storage level (<code>MEMORY_AND_DISK</code>).
<code>DataFrame</code>	<code>persist(StorageLevel newLevel)</code> Persist this <code>DataFrame</code> with the given storage level.
<code>void</code>	<code>printSchema()</code> Prints the schema to the console in a nice tree format.
<code>org.apache.spark.sql.execution.QueryExecution</code>	<code>queryExecution()</code>

DataFrame[]	<code>randomSplit(double[] weights)</code> Randomly splits this <code>DataFrame</code> with the provided weights.
DataFrame[]	<code>randomSplit(double[] weights, long seed)</code> Randomly splits this <code>DataFrame</code> with the provided weights.
RDD<Row>	<code>rdd()</code> Represents the content of the <code>DataFrame</code> as an RDD of Rows.
void	<code>registerTempTable(java.lang.String tableName)</code> Registers this <code>DataFrame</code> as a temporary table using the given name.
DataFrame	<code>repartition(Column... partitionExprs)</code> Returns a new <code>DataFrame</code> partitioned by the given partitioning expressions preserving the partitions.
DataFrame	<code>repartition(int numPartitions)</code> Returns a new <code>DataFrame</code> that has exactly <code>numPartitions</code> partitions.
DataFrame	<code>repartition(int numPartitions, Column... partitionExprs)</code> Returns a new <code>DataFrame</code> partitioned by the given partitioning expressions into <code>numPartitions</code> partitions.
DataFrame	<code>repartition(int numPartitions, scala.collection.Seq<Column> partitionExprs)</code> Returns a new <code>DataFrame</code> partitioned by the given partitioning expressions into <code>numPartitions</code> partitions.
DataFrame	<code>repartition(scala.collection.Seq<Column> partitionExprs)</code> Returns a new <code>DataFrame</code> partitioned by the given partitioning expressions preserving the partitions.
protected org.apache.spark.sql.catalyst.expressions.NamedExpression	<code>resolve(java.lang.String colName)</code>
GroupedData	<code>rollup(Column... cols)</code> Create a multi-dimensional rollup for the current <code>DataFrame</code> using the specified columns, aggregation on them.
GroupedData	<code>rollup(scala.collection.Seq<Column> cols)</code> Create a multi-dimensional rollup for the current <code>DataFrame</code> using the specified columns, aggregation on them.
GroupedData	<code>rollup(java.lang.String col1, scala.collection.Seq<java.lang.String> cols)</code> Create a multi-dimensional rollup for the current <code>DataFrame</code> using the specified columns, aggregation on them.
GroupedData	<code>rollup(java.lang.String col1, java.lang.String... cols)</code> Create a multi-dimensional rollup for the current <code>DataFrame</code> using the specified columns, aggregation on them.
DataFrame	<code>sample(boolean withReplacement, double fraction)</code> Returns a new <code>DataFrame</code> by sampling a fraction of rows, using a random seed.
DataFrame	<code>sample(boolean withReplacement, double fraction, long seed)</code> Returns a new <code>DataFrame</code> by sampling a fraction of rows.
void	<code>save(java.lang.String path)</code> Deprecated. As of 1.4.0, replaced by <code>write().save(path)</code> . This will be removed in Spark 2.0.
void	<code>save(java.lang.String path, SaveMode mode)</code> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).save(path)</code> . This will be removed in Spark 2.0.
void	<code>save(java.lang.String source, SaveMode mode, java.util.Map<java.lang.String, java.lang.String> options)</code> Deprecated. As of 1.4.0, replaced by <code>write().format(source).mode(mode).options(options).save()</code> . This will be removed in Spark 2.0.
void	<code>save(java.lang.String source, SaveMode mode, scala.collection.immutable.Map<java.lang.String, java.lang.String> options)</code> Deprecated. As of 1.4.0, replaced by <code>write().format(source).mode(mode).options(options).save()</code> . This will be removed in Spark 2.0.
void	<code>save(java.lang.String path, java.lang.String source)</code> Deprecated. As of 1.4.0, replaced by <code>write().format(source).save(path)</code> . This will be removed in Spark 2.0.
void	<code>saveAsParquetFile(java.lang.String path)</code> Deprecated. As of 1.4.0, replaced by <code>write().parquet()</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName)</code> Deprecated. As of 1.4.0, replaced by <code>write().saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName, SaveMode mode)</code> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName, java.lang.String source)</code> Deprecated. As of 1.4.0, replaced by <code>write().format(source).saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName, java.lang.String source, SaveMode mode)</code> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).format(source).saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName, java.lang.String source, SaveMode mode, java.util.Map<java.lang.String, java.lang.String> options)</code> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).format(source).options(options).saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
void	<code>saveAsTable(java.lang.String tableName, java.lang.String source, SaveMode mode, scala.collection.immutable.Map<java.lang.String, java.lang.String> options)</code> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).format(source).options(options).saveAsTable(tableName)</code> . This will be removed in Spark 2.0.
StructType	<code>schema()</code> Returns the schema of this <code>DataFrame</code> .

DataFrame	<code>select(Column... cols)</code> Selects a set of column based expressions.
DataFrame	<code>select(scala.collection.Seq<Column> cols)</code> Selects a set of column based expressions.
DataFrame	<code>select(java.lang.String col, scala.collection.Seq<java.lang.String> cols)</code> Selects a set of columns.
DataFrame	<code>select(java.lang.String col, java.lang.String... cols)</code> Selects a set of columns.
DataFrame	<code>selectExpr(scala.collection.Seq<java.lang.String> exprs)</code> Selects a set of SQL expressions.
DataFrame	<code>selectExpr(java.lang.String... exprs)</code> Selects a set of SQL expressions.
void	<code>show()</code> Displays the top 20 rows of DataFrame in a tabular form.
void	<code>show(boolean truncate)</code> Displays the top 20 rows of DataFrame in a tabular form.
void	<code>show(int numRows)</code> Displays the DataFrame in a tabular form.
void	<code>show(int numRows, boolean truncate)</code> Displays the DataFrame in a tabular form.
DataFrame	<code>sort(Column... sortExprs)</code> Returns a new DataFrame sorted by the given expressions.
DataFrame	<code>sort(scala.collection.Seq<Column> sortExprs)</code> Returns a new DataFrame sorted by the given expressions.
DataFrame	<code>sort(java.lang.String sortCol, scala.collection.Seq<java.lang.String> sortCols)</code> Returns a new DataFrame sorted by the specified column, all in ascending order.
DataFrame	<code>sort(java.lang.String sortCol, java.lang.String... sortCols)</code> Returns a new DataFrame sorted by the specified column, all in ascending order.
DataFrame	<code>sortWithinPartitions(Column... sortExprs)</code> Returns a new DataFrame with each partition sorted by the given expressions.
DataFrame	<code>sortWithinPartitions(scala.collection.Seq<Column> sortExprs)</code> Returns a new DataFrame with each partition sorted by the given expressions.
DataFrame	<code>sortWithinPartitions(java.lang.String sortCol, scala.collection.Seq<java.lang.String> sortCols)</code> Returns a new DataFrame with each partition sorted by the given expressions.
DataFrame	<code>sortWithinPartitions(java.lang.String sortCol, java.lang.String... sortCols)</code> Returns a new DataFrame with each partition sorted by the given expressions.
SQLContext	<code>sqlContext()</code>
DataFrameStatFunctions	<code>stat()</code> Returns a DataFrameStatFunctions for working statistic functions support.
Row[]	<code>take(int n)</code> Returns the first <i>n</i> rows in the DataFrame .
java.util.List<Row>	<code>takeAsList(int n)</code> Returns the first <i>n</i> rows in the DataFrame as a list.
DataFrame	<code>toDF()</code> Returns the object itself.
DataFrame	<code>toDF(scala.collection.Seq<java.lang.String> colNames)</code> Returns a new DataFrame with columns renamed.
DataFrame	<code>toDF(java.lang.String... colNames)</code> Returns a new DataFrame with columns renamed.
JavaRDD<Row>	<code>toJavaRDD()</code> Returns the content of the DataFrame as a JavaRDD of Rows.
RDD<java.lang.String>	<code>toJSON()</code> Returns the content of the DataFrame as a RDD of JSON strings.
DataFrame	<code>toSchemaRDD()</code> Deprecated. <i>As of 1.3.0, replaced by toDF(). This will be removed in Spark 2.0.</i>
<U> DataFrame	<code>transform(scala.Function1<DataFrame, DataFrame> t)</code> Concise syntax for chaining custom transformations.
DataFrame	<code>unionAll(DataFrame other)</code> Returns a new DataFrame containing union of rows in this frame and another frame.
DataFrame	<code>unpersist()</code> Mark the DataFrame as non-persistent, and remove all blocks for it from memory and disk.
DataFrame	<code>unpersist(boolean blocking)</code> Mark the DataFrame as non-persistent, and remove all blocks for it from memory and disk.
DataFrame	<code>where(Column condition)</code> Filters rows using the given condition.
DataFrame	<code>where(java.lang.String conditionExpr)</code> Filters rows using the given SQL expression.
DataFrame	<code>withColumn(java.lang.String colName, Column col)</code> Returns a new DataFrame by adding a column or replacing the existing column that has the same name.
DataFrame	<code>withColumnRenamed(java.lang.String existingName, java.lang.String newName)</code> Returns a new DataFrame with a column renamed.
DataFrameWriter	<code>write()</code> Interface for saving the content of the DataFrame out into external storage.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods inherited from interface org.apache.spark.sql.execution.Queryable

toString

Constructor Detail

DataFrame

```
public DataFrame(SQLContext sqlContext,
    org.apache.spark.sql.catalyst.plans.logical.LogicalPlan logicalPlan)
```

A constructor that automatically analyzes the logical plan.

This reports error eagerly as the `DataFrame` is constructed, unless `SQLConf.dataFrameEagerAnalysis` is turned off.

Parameters:

- `sqlContext` - (undocumented)
- `logicalPlan` - (undocumented)

Method Detail

toDF

```
public DataFrame toDF(java.lang.String... colNames)
```

Returns a new `DataFrame` with columns renamed. This can be quite convenient in conversion from a RDD of tuples into a `DataFrame` with meaningful names. For example:

```
val rdd: RDD[(Int, String)] = ...
rdd.toDF() // this implicit conversion creates a DataFrame with column name _1 and _2
rdd.toDF("id", "name") // this creates a DataFrame with column name "id" and "name"
```

Parameters:

- `colNames` - (undocumented)

Returns:

- (undocumented)

Since:

- 1.3.0

sortWithinPartitions

```
public DataFrame sortWithinPartitions(java.lang.String sortCol,
    java.lang.String... sortCols)
```

Returns a new `DataFrame` with each partition sorted by the given expressions.

This is the same operation as "SORT BY" in SQL (Hive QL).

Parameters:

- `sortCol` - (undocumented)
- `sortCols` - (undocumented)

Returns:

- (undocumented)

Since:

- 1.6.0

sortWithinPartitions

```
public DataFrame sortWithinPartitions(Column... sortExprs)
```

Returns a new `DataFrame` with each partition sorted by the given expressions.

This is the same operation as "SORT BY" in SQL (Hive QL).

Parameters:

- `sortExprs` - (undocumented)

Returns:

- (undocumented)

Since:

- 1.6.0

sort

```
public DataFrame sort(java.lang.String sortCol,
    java.lang.String... sortCols)
```

Returns a new `DataFrame` sorted by the specified column, all in ascending order.

```
// The following 3 are equivalent
df.sort("sortcol")
df.sort($"sortcol")
```

```
df.sort($"sortcol".asc)
```

Parameters:

- sortCol - (undocumented)
- sortCols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

sort

```
public DataFrame sort(Column... sortExprs)
```

Returns a new `DataFrame` sorted by the given expressions. For example:

```
df.sort($"col1", $"col2".desc)
```

Parameters:

- sortExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

orderBy

```
public DataFrame orderBy(java.lang.String sortCol,
                          java.lang.String... sortCols)
```

Returns a new `DataFrame` sorted by the given expressions. This is an alias of the `sort` function.

Parameters:

- sortCol - (undocumented)
- sortCols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

orderBy

```
public DataFrame orderBy(Column... sortExprs)
```

Returns a new `DataFrame` sorted by the given expressions. This is an alias of the `sort` function.

Parameters:

- sortExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

select

```
public DataFrame select(Column... cols)
```

Selects a set of column based expressions.

```
df.select($"colA", $"colB" + 1)
```

Parameters:

- cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

select

```
public DataFrame select(java.lang.String col,
```

```
java.lang.String... cols)
```

Selects a set of columns. This is a variant of `select` that can only select existing columns using column names (i.e. cannot construct expressions).

```
// The following two are equivalent:
df.select("colA", "colB")
df.select($"colA", $"colB")
```

Parameters:

`col` - (undocumented)
`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

selectExpr

```
public DataFrame selectExpr(java.lang.String... exprs)
```

Selects a set of SQL expressions. This is a variant of `select` that accepts SQL expressions.

```
// The following are equivalent:
df.selectExpr("colA", "colB as newName", "abs(colC)")
df.select(expr("colA"), expr("colB as newName"), expr("abs(colC)"))
```

Parameters:

`exprs` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

groupBy

```
public GroupedData groupBy(Column... cols)
```

Groups the `DataFrame` using the specified columns, so we can run aggregation on them. See `GroupedData` for all the available aggregate functions.

```
// Compute the average for all numeric columns grouped by department.
df.groupBy($"department").avg()

// Compute the max age and average salary, grouped by department and gender.
df.groupBy($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

rollup

```
public GroupedData rollup(Column... cols)
```

Create a multi-dimensional rollup for the current `DataFrame` using the specified columns, so we can run aggregation on them. See `GroupedData` for all the available aggregate functions.

```
// Compute the average for all numeric columns rolled up by department and group.
df.rollup($"department", $"group").avg()

// Compute the max age and average salary, rolled up by department and gender.
df.rollup($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

cube

```
public GroupedData cube(Column... cols)
```

Create a multi-dimensional cube for the current `DataFrame` using the specified columns, so we can run aggregation on them. See `GroupedData` for all the available aggregate functions.

```
// Compute the average for all numeric columns cubed by department and group.
df.cube($"department", $"group").avg()

// Compute the max age and average salary, cubed by department and gender.
df.cube($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

groupBy

```
public GroupedData groupBy(java.lang.String col1,
                             java.lang.String... cols)
```

Groups the `DataFrame` using the specified columns, so we can run aggregation on them. See `GroupedData` for all the available aggregate functions.

This is a variant of `groupBy` that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns grouped by department.
df.groupBy("department").avg()

// Compute the max age and average salary, grouped by department and gender.
df.groupBy($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`col1` - (undocumented)

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

rollup

```
public GroupedData rollup(java.lang.String col1,
                             java.lang.String... cols)
```

Create a multi-dimensional rollup for the current `DataFrame` using the specified columns, so we can run aggregation on them. See `GroupedData` for all the available aggregate functions.

This is a variant of `rollup` that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns rolled up by department and group.
df.rollup("department", "group").avg()

// Compute the max age and average salary, rolled up by department and gender.
df.rollup($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`col1` - (undocumented)

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

cube

```
public GroupedData cube(java.lang.String col1,
                        java.lang.String... cols)
```

Create a multi-dimensional cube for the current [DataFrame](#) using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

This is a variant of cube that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns cubed by department and group.
df.cube("department", "group").avg()

// Compute the max age and average salary, cubed by department and gender.
df.cube($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

col1 - (undocumented)

cols - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

agg

```
public DataFrame agg(Column expr,
                    Column... exprs)
```

Aggregates on the entire [DataFrame](#) without groups.

```
// df.agg(...) is a shorthand for df.groupBy().agg(...)
df.agg(max($"age"), avg($"salary"))
df.groupBy().agg(max($"age"), avg($"salary"))
```

Parameters:

expr - (undocumented)

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

describe

```
public DataFrame describe(java.lang.String... cols)
```

Computes statistics for numeric columns, including count, mean, stddev, min, and max. If no columns are given, this function computes statistics for all numerical columns.

This function is meant for exploratory data analysis, as we make no guarantee about the backward compatibility of the schema of the resulting [DataFrame](#). If you want to programmatically compute summary statistics, use the [agg](#) function instead.

```
df.describe("age", "height").show()

// output:
// summary age    height
// count   10.0   10.0
// mean    53.3   178.05
// stddev  11.6   15.7
// min     18.0   163.0
// max     92.0   192.0
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.1

repartition

```
public DataFrame repartition(int numPartitions,
                             Column... partitionExprs)
```

Returns a new `DataFrame` partitioned by the given partitioning expressions into `numPartitions`. The resulting `DataFrame` is hash partitioned.

This is the same operation as "DISTRIBUTE BY" in SQL (Hive QL).

Parameters:

- `numPartitions` - (undocumented)
- `partitionExprs` - (undocumented)

Returns:

- (undocumented)

Since:

- 1.6.0

repartition

```
public DataFrame repartition(Column... partitionExprs)
```

Returns a new `DataFrame` partitioned by the given partitioning expressions preserving the existing number of partitions. The resulting `DataFrame` is hash partitioned.

This is the same operation as "DISTRIBUTE BY" in SQL (Hive QL).

Parameters:

- `partitionExprs` - (undocumented)

Returns:

- (undocumented)

Since:

- 1.6.0

sqlContext

```
public SQLContext sqlContext()
```

Specified by:

- `sqlContext` in interface `org.apache.spark.sql.execution.Queryable`

queryExecution

```
public org.apache.spark.sql.execution.QueryExecution queryExecution()
```

Specified by:

- `queryExecution` in interface `org.apache.spark.sql.execution.Queryable`

logicalPlan

```
protected org.apache.spark.sql.catalyst.plans.logical.LogicalPlan logicalPlan()
```

resolve

```
protected org.apache.spark.sql.catalyst.expressions.NamedExpression resolve(java.lang.String colName)
```

numericColumns

```
protected scala.collection.Seq<org.apache.spark.sql.catalyst.expressions.Expression> numericColumns()
```

toDF

```
public DataFrame toDF()
```

Returns the object itself.

Returns:

- (undocumented)

Since:

- 1.3.0

as

Experimental

```
public <U> Dataset<U> as(Encoder<U> evidence$1)
```

Converts this `DataFrame` to a strongly-typed `Dataset` containing objects of the specified type, `U`.

Parameters:

- `evidence$1` - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

toDF

```
public DataFrame toDF(Scala.collection.Seq<java.lang.String> colNames)
```

Returns a new `DataFrame` with columns renamed. This can be quite convenient in conversion from a `RDD` of tuples into a `DataFrame` with meaningful names. For example:

```
val rdd: RDD[(Int, String)] = ...
rdd.toDF() // this implicit conversion creates a DataFrame with column name _1 and _2
rdd.toDF("id", "name") // this creates a DataFrame with column name "id" and "name"
```

Parameters:`colNames` - (undocumented)**Returns:**

(undocumented)

Since:

1.3.0

schema

```
public StructType schema()
```

Returns the schema of this `DataFrame`.

Specified by:`schema` in interface `org.apache.spark.sql.execution.Queryable`**Returns:**

(undocumented)

Since:

1.3.0

printSchema

```
public void printSchema()
```

Prints the schema to the console in a nice tree format.

Specified by:`printSchema` in interface `org.apache.spark.sql.execution.Queryable`**Since:**

1.3.0

explain

```
public void explain(boolean extended)
```

Prints the plans (logical and physical) to the console for debugging purposes.

Specified by:`explain` in interface `org.apache.spark.sql.execution.Queryable`**Parameters:**`extended` - (undocumented)**Since:**

1.3.0

explain

```
public void explain()
```

Prints the physical plan to the console for debugging purposes.

Specified by:`explain` in interface `org.apache.spark.sql.execution.Queryable`**Since:**

1.3.0

dtypes

```
public scala.Tuple2<java.lang.String, java.lang.String>[] dtypes()
```

Returns all column names and their data types as an array.

Returns:

(undocumented)

Since:

1.3.0

columns

public java.lang.String[] columns()

Returns all column names as an array.

Returns:

(undocumented)

Since:

1.3.0

isLocal

public boolean isLocal()

Returns true if the collect and take methods can be run locally (without any Spark executors).

Returns:

(undocumented)

Since:

1.3.0

show

public void show(int numRows)

Displays the `DataFrame` in a tabular form. Strings more than 20 characters will be truncated, and all cells will be aligned right. For example:

year	month	AVG('Adj Close)	MAX('Adj Close)
1980	12	0.503218	0.595103
1981	01	0.523289	0.570307
1982	02	0.436504	0.475256
1983	03	0.410516	0.442194
1984	04	0.450090	0.483521

Parameters:

numRows - Number of rows to show

Since:

1.3.0

show

public void show()

Displays the top 20 rows of `DataFrame` in a tabular form. Strings more than 20 characters will be truncated, and all cells will be aligned right.

Since:

1.3.0

show

public void show(boolean truncate)

Displays the top 20 rows of `DataFrame` in a tabular form.

Parameters:

truncate - Whether truncate long strings. If true, strings more than 20 characters will be truncated and all cells will be aligned right

Since:

1.5.0

show

public void show(int numRows,
boolean truncate)

Displays the `DataFrame` in a tabular form. For example:

year	month	AVG('Adj Close)	MAX('Adj Close)
1980	12	0.503218	0.595103
1981	01	0.523289	0.570307

1982	02	0.436504	0.475256
1983	03	0.410516	0.442194
1984	04	0.450090	0.483521

Parameters:

numRows - Number of rows to show

truncate - Whether truncate long strings. If true, strings more than 20 characters will be truncated and all cells will be aligned right

Since:

1.5.0

na

```
public DataFrameNaFunctions na()
```

Returns a `DataFrameNaFunctions` for working with missing data.

```
// Dropping rows containing any null values.
df.na.drop()
```

Returns:

(undocumented)

Since:

1.3.1

stat

```
public DataFrameStatFunctions stat()
```

Returns a `DataFrameStatFunctions` for working statistic functions support.

```
// Finding frequent items in column with name 'a'.
df.stat.freqItems(Seq("a"))
```

Returns:

(undocumented)

Since:

1.4.0

join

```
public DataFrame join(DataFrame right)
```

Cartesian join with another `DataFrame`.

Note that cartesian joins are very expensive without an extra filter that can be pushed down.

Parameters:

right - Right side of the join operation.

Returns:

(undocumented)

Since:

1.3.0

join

```
public DataFrame join(DataFrame right,
                      java.lang.String usingColumn)
```

Inner equi-join with another `DataFrame` using the given column.

Different from other join functions, the join column will only appear once in the output, i.e. similar to SQL's `JOIN USING` syntax.

```
// Joining df1 and df2 using the column "user_id"
df1.join(df2, "user_id")
```

Note that if you perform a self-join using this function without aliasing the input `DataFrames`, you will NOT be able to reference any columns after the join, since there is no way to disambiguate which side of the join you would like to reference.

Parameters:

right - Right side of the join operation.

usingColumn - Name of the column to join on. This column must exist on both sides.

Returns:

(undocumented)

Since:

1.4.0

join

```
public DataFrame join(DataFrame right,
    scala.collection.Seq<java.lang.String> usingColumns)
```

Inner equi-join with another `DataFrame` using the given columns.

Different from other join functions, the join columns will only appear once in the output, i.e. similar to SQL's JOIN USING syntax.

```
// Joining df1 and df2 using the columns "user_id" and "user_name"
df1.join(df2, Seq("user_id", "user_name"))
```

Note that if you perform a self-join using this function without aliasing the input `DataFrames`, you will NOT be able to reference any columns after the join, since there is no way to disambiguate which side of the join you would like to reference.

Parameters:

`right` - Right side of the join operation.

`usingColumns` - Names of the columns to join on. This columns must exist on both sides.

Returns:

(undocumented)

Since:

1.4.0

join

```
public DataFrame join(DataFrame right,
    scala.collection.Seq<java.lang.String> usingColumns,
    java.lang.String joinType)
```

Equi-join with another `DataFrame` using the given columns.

Different from other join functions, the join columns will only appear once in the output, i.e. similar to SQL's JOIN USING syntax.

Note that if you perform a self-join using this function without aliasing the input `DataFrames`, you will NOT be able to reference any columns after the join, since there is no way to disambiguate which side of the join you would like to reference.

Parameters:

`right` - Right side of the join operation.

`usingColumns` - Names of the columns to join on. This columns must exist on both sides.

`joinType` - One of: inner, outer, left_outer, right_outer, leftsemi.

Returns:

(undocumented)

Since:

1.6.0

join

```
public DataFrame join(DataFrame right,
    Column joinExprs)
```

Inner join with another `DataFrame`, using the given join expression.

```
// The following two are equivalent:
df1.join(df2, $"df1Key" === $"df2Key")
df1.join(df2).where($"df1Key" === $"df2Key")
```

Parameters:

`right` - (undocumented)

`joinExprs` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

join

```
public DataFrame join(DataFrame right,
    Column joinExprs,
    java.lang.String joinType)
```

Join with another `DataFrame`, using the given join expression. The following performs a full outer join between `df1` and `df2`.

```
// Scala:
import org.apache.spark.sql.functions._
df1.join(df2, $"df1Key" === $"df2Key", "outer")
```

```
// Java:
import static org.apache.spark.sql.functions.*;
df1.join(df2, col("df1Key").equalTo(col("df2Key")), "outer");
```

Parameters:

- right - Right side of the join.
- joinExprs - Join expression.
- joinType - One of: inner, outer, left_outer, right_outer, leftsemi.

Returns:

(undocumented)

Since:

1.3.0

sortWithinPartitions

```
public DataFrame sortWithinPartitions(java.lang.String sortCol,
                                     scala.collection.Seq<java.lang.String> sortCols)
```

Returns a new `DataFrame` with each partition sorted by the given expressions.

This is the same operation as "SORT BY" in SQL (Hive QL).

Parameters:

- sortCol - (undocumented)
- sortCols - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

sortWithinPartitions

```
public DataFrame sortWithinPartitions(scala.collection.Seq<Column> sortExprs)
```

Returns a new `DataFrame` with each partition sorted by the given expressions.

This is the same operation as "SORT BY" in SQL (Hive QL).

Parameters:

- sortExprs - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

sort

```
public DataFrame sort(java.lang.String sortCol,
                     scala.collection.Seq<java.lang.String> sortCols)
```

Returns a new `DataFrame` sorted by the specified column, all in ascending order.

```
// The following 3 are equivalent
df.sort("sortcol")
df.sort($"sortcol")
df.sort($"sortcol".asc)
```

Parameters:

- sortCol - (undocumented)
- sortCols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

sort

```
public DataFrame sort(scala.collection.Seq<Column> sortExprs)
```

Returns a new `DataFrame` sorted by the given expressions. For example:

```
df.sort($"col1", $"col2".desc)
```

Parameters:

sortExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

orderBy

public DataFrame orderBy(java.lang.String sortCol,scala.collection.Seq<java.lang.String> sortCols)

Returns a new DataFrame sorted by the given expressions. This is an alias of the sort function.

Parameters:

sortCol - (undocumented)

sortCols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

orderBy

public DataFrame orderBy(scala.collection.Seq<Column> sortExprs)

Returns a new DataFrame sorted by the given expressions. This is an alias of the sort function.

Parameters:

sortExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

apply

public Column apply(java.lang.String colName)

Selects column based on the column name and return it as a Column. Note that the column name can also reference to a nested column like a.b.

Parameters:

colName - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

col

public Column col(java.lang.String colName)

Selects column based on the column name and return it as a Column. Note that the column name can also reference to a nested column like a.b.

Parameters:

colName - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

as

public DataFrame as(java.lang.String alias)

Returns a new DataFrame with an alias set.

Parameters:

alias - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

as

```
public DataFrame as(scala.Symbol alias)
```

(Scala-specific) Returns a new `DataFrame` with an alias set.

Parameters:

alias - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

alias

```
public DataFrame alias(java.lang.String alias)
```

Returns a new `DataFrame` with an alias set. Same as `as`.

Parameters:

alias - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

alias

```
public DataFrame alias(scala.Symbol alias)
```

(Scala-specific) Returns a new `DataFrame` with an alias set. Same as `as`.

Parameters:

alias - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

select

```
public DataFrame select(scala.collection.Seq<Column> cols)
```

Selects a set of column based expressions.

```
df.select($"colA", $"colB" + 1)
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

select

```
public DataFrame select(java.lang.String col,
                        scala.collection.Seq<java.lang.String> cols)
```

Selects a set of columns. This is a variant of `select` that can only select existing columns using column names (i.e. cannot construct expressions).

```
// The following two are equivalent:
df.select("colA", "colB")
df.select($"colA", $"colB")
```

Parameters:

col - (undocumented)

cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

selectExpr

public DataFrame selectExpr(Scala.collection.Seq<java.lang.String> exprs)

Selects a set of SQL expressions. This is a variant of select that accepts SQL expressions.

```
// The following are equivalent:
df.selectExpr("colA", "colB as newName", "abs(colC)")
df.select(expr("colA"), expr("colB as newName"), expr("abs(colC)"))
```

Parameters:

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

filter

public DataFrame filter(Column condition)

Filters rows using the given condition.

```
// The following are equivalent:
peopleDf.filter($"age" > 15)
peopleDf.where($"age" > 15)
```

Parameters:

condition - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

filter

public DataFrame filter(java.lang.String conditionExpr)

Filters rows using the given SQL expression.

```
peopleDf.filter("age > 15")
```

Parameters:

conditionExpr - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

where

public DataFrame where(Column condition)

Filters rows using the given condition. This is an alias for filter.

```
// The following are equivalent:
peopleDf.filter($"age" > 15)
peopleDf.where($"age" > 15)
```

Parameters:

condition - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

where

public DataFrame where(java.lang.String conditionExpr)

Filters rows using the given SQL expression.

```
peopleDf.where("age > 15")
```

Parameters:

conditionExpr - (undocumented)

Returns:

(undocumented)

Since:

1.5.0

groupBy

```
public GroupedData groupBy(scala.collection.Seq<Column> cols)
```

Groups the [DataFrame](#) using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

```
// Compute the average for all numeric columns grouped by department.
df.groupBy($"department").avg()

// Compute the max age and average salary, grouped by department and gender.
df.groupBy($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

rollup

```
public GroupedData rollup(scala.collection.Seq<Column> cols)
```

Create a multi-dimensional rollup for the current [DataFrame](#) using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

```
// Compute the average for all numeric columns rolled up by department and group.
df.rollup($"department", $"group").avg()

// Compute the max age and average salary, rolled up by department and gender.
df.rollup($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

cube

```
public GroupedData cube(scala.collection.Seq<Column> cols)
```

Create a multi-dimensional cube for the current [DataFrame](#) using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

```
// Compute the average for all numeric columns cubed by department and group.
df.cube($"department", $"group").avg()

// Compute the max age and average salary, cubed by department and gender.
df.cube($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

groupBy

```
public GroupedData groupBy(java.lang.String col1,
                           scala.collection.Seq<java.lang.String> cols)
```

Groups the `DataFrame` using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

This is a variant of `groupBy` that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns grouped by department.
df.groupBy("department").avg()

// Compute the max age and average salary, grouped by department and gender.
df.groupBy($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`col1` - (undocumented)

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

rollup

```
public GroupedData rollup(java.lang.String col1,
                           scala.collection.Seq<java.lang.String> cols)
```

Create a multi-dimensional rollup for the current `DataFrame` using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

This is a variant of `rollup` that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns rolled up by department and group.
df.rollup("department", "group").avg()

// Compute the max age and average salary, rolled up by department and gender.
df.rollup($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`col1` - (undocumented)

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

cube

```
public GroupedData cube(java.lang.String col1,
                         scala.collection.Seq<java.lang.String> cols)
```

Create a multi-dimensional cube for the current `DataFrame` using the specified columns, so we can run aggregation on them. See [GroupedData](#) for all the available aggregate functions.

This is a variant of `cube` that can only group by existing columns using column names (i.e. cannot construct expressions).

```
// Compute the average for all numeric columns cubed by department and group.
df.cube("department", "group").avg()

// Compute the max age and average salary, cubed by department and gender.
df.cube($"department", $"gender").agg(Map(
  "salary" -> "avg",
  "age" -> "max"
))
```

Parameters:

`col1` - (undocumented)

`cols` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

agg

```
public DataFrame agg(scala.Tuple2<java.lang.String,java.lang.String> aggExpr,
    scala.collection.Seq<scala.Tuple2<java.lang.String,java.lang.String>> aggExprs)
```

(Scala-specific) Aggregates on the entire `DataFrame` without groups.

```
// df.agg(...) is a shorthand for df.groupBy().agg(...)
df.agg("age" -> "max", "salary" -> "avg")
df.groupBy().agg("age" -> "max", "salary" -> "avg")
```

Parameters:

aggExpr - (undocumented)

aggExprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(scala.collection.immutable.Map<java.lang.String,java.lang.String> exprs)
```

(Scala-specific) Aggregates on the entire `DataFrame` without groups.

```
// df.agg(...) is a shorthand for df.groupBy().agg(...)
df.agg(Map("age" -> "max", "salary" -> "avg"))
df.groupBy().agg(Map("age" -> "max", "salary" -> "avg"))
```

Parameters:

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(java.util.Map<java.lang.String,java.lang.String> exprs)
```

(Java-specific) Aggregates on the entire `DataFrame` without groups.

```
// df.agg(...) is a shorthand for df.groupBy().agg(...)
df.agg(Map("age" -> "max", "salary" -> "avg"))
df.groupBy().agg(Map("age" -> "max", "salary" -> "avg"))
```

Parameters:

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

agg

```
public DataFrame agg(Column expr,
    scala.collection.Seq<Column> exprs)
```

Aggregates on the entire `DataFrame` without groups.

```
// df.agg(...) is a shorthand for df.groupBy().agg(...)
df.agg(max($"age"), avg($"salary"))
df.groupBy().agg(max($"age"), avg($"salary"))
```

Parameters:

expr - (undocumented)

exprs - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

limit

```
public DataFrame limit(int n)
```

Returns a new `DataFrame` by taking the first `n` rows. The difference between this function and `head` is that `head` returns an array while `limit` returns a new `DataFrame`.

Parameters:

`n` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

unionAll

```
public DataFrame unionAll(DataFrame other)
```

Returns a new `DataFrame` containing union of rows in this frame and another frame. This is equivalent to `UNION ALL` in SQL.

Parameters:

`other` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

intersect

```
public DataFrame intersect(DataFrame other)
```

Returns a new `DataFrame` containing rows only in both this frame and another frame. This is equivalent to `INTERSECT` in SQL.

Parameters:

`other` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

except

```
public DataFrame except(DataFrame other)
```

Returns a new `DataFrame` containing rows in this frame but not in another frame. This is equivalent to `EXCEPT` in SQL.

Parameters:

`other` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

sample

```
public DataFrame sample(boolean withReplacement,
                        double fraction,
                        long seed)
```

Returns a new `DataFrame` by sampling a fraction of rows.

Parameters:

`withReplacement` - Sample with replacement or not.

`fraction` - Fraction of rows to generate.

`seed` - Seed for sampling.

Returns:

(undocumented)

Since:

1.3.0

sample

```
public DataFrame sample(boolean withReplacement,
                        double fraction)
```

Returns a new `DataFrame` by sampling a fraction of rows, using a random seed.

Parameters:

- `withReplacement` - Sample with replacement or not.
- `fraction` - Fraction of rows to generate.

Returns:

(undocumented)

Since:

1.3.0

randomSplit

```
public DataFrame[] randomSplit(double[] weights,
                               long seed)
```

Randomly splits this `DataFrame` with the provided weights.

Parameters:

- `weights` - weights for splits, will be normalized if they don't sum to 1.
- `seed` - Seed for sampling.

Returns:

(undocumented)

Since:

1.4.0

randomSplit

```
public DataFrame[] randomSplit(double[] weights)
```

Randomly splits this `DataFrame` with the provided weights.

Parameters:

- `weights` - weights for splits, will be normalized if they don't sum to 1.

Returns:

(undocumented)

Since:

1.4.0

explode

```
public <A extends scala.Product> DataFrame explode(
    scala.collection.Seq<Column> input,
    scala.Function1<Row, scala.collection.TraversableOnce<A>> f,
    scala.reflect.api.TypeTags.TypeTag<A> evidence$2)
```

(Scala-specific) Returns a new `DataFrame` where each row has been expanded to zero or more rows by the provided function. This is similar to a `LATERAL VIEW` in HiveQL. The columns of the input row are implicitly joined with each row that is output by the function.

The following example uses this function to count the number of books which contain a given word:

```
case class Book(title: String, words: String)
val df: RDD[Book]

case class Word(word: String)
val allWords = df.explode('words) {
  case Row(words: String) => words.split(" ").map(Word(_))
}

val bookCountPerWord = allWords.groupBy("word").agg(countDistinct("title"))
```

Parameters:

- `input` - (undocumented)
- `f` - (undocumented)
- `evidence$2` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

explode

```
public <A,B> DataFrame explode(
    java.lang.String inputColumn,
    java.lang.String outputColumn,
```



```
scala.Function1<A,scala.collection.TraversableOnce<B>> f,  
scala.reflect.api.TypeTags.TypeTag<B> evidence$3)
```

(Scala-specific) Returns a new `DataFrame` where a single column has been expanded to zero or more rows by the provided function. This is similar to a `LATERAL VIEW` in HiveQL. All columns of the input row are implicitly joined with each value that is output by the function.

```
df.explode("words", "word"){words: String => words.split(" ")}
```

Parameters:

- inputColumn - (undocumented)
- outputColumn - (undocumented)
- f - (undocumented)
- evidence\$3 - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

withColumn

```
public DataFrame withColumn(java.lang.String colName,  
                             Column col)
```

Returns a new `DataFrame` by adding a column or replacing the existing column that has the same name.

Parameters:

- colName - (undocumented)
- col - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

withColumnRenamed

```
public DataFrame withColumnRenamed(java.lang.String existingName,  
                                    java.lang.String newName)
```

Returns a new `DataFrame` with a column renamed. This is a no-op if schema doesn't contain existingName.

Parameters:

- existingName - (undocumented)
- newName - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

drop

```
public DataFrame drop(java.lang.String colName)
```

Returns a new `DataFrame` with a column dropped. This is a no-op if schema doesn't contain column name.

Parameters:

- colName - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

drop

```
public DataFrame drop(Column col)
```

Returns a new `DataFrame` with a column dropped. This version of drop accepts a `Column` rather than a name. This is a no-op if the `DataFrame` doesn't have a column with an equivalent expression.

Parameters:

- col - (undocumented)

Returns:

(undocumented)

Since:

1.4.1

dropDuplicates

```
public DataFrame dropDuplicates()
```

Returns a new `DataFrame` that contains only the unique rows from this `DataFrame`. This is an alias for `distinct`.

Returns:

(undocumented)

Since:

1.4.0

dropDuplicates

```
public DataFrame dropDuplicates(Scala.collection.Seq<java.lang.String> colNames)
```

(Scala-specific) Returns a new `DataFrame` with duplicate rows removed, considering only the subset of columns.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

dropDuplicates

```
public DataFrame dropDuplicates(java.lang.String[] colNames)
```

Returns a new `DataFrame` with duplicate rows removed, considering only the subset of columns.

Parameters:

colNames - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

describe

```
public DataFrame describe(Scala.collection.Seq<java.lang.String> cols)
```

Computes statistics for numeric columns, including count, mean, stddev, min, and max. If no columns are given, this function computes statistics for all numerical columns.

This function is meant for exploratory data analysis, as we make no guarantee about the backward compatibility of the schema of the resulting `DataFrame`. If you want to programmatically compute summary statistics, use the `agg` function instead.

```
df.describe("age", "height").show()
```

```
// output:
// summary age    height
// count   10.0   10.0
// mean    53.3   178.05
// stddev  11.6   15.7
// min     18.0   163.0
// max     92.0   192.0
```

Parameters:

cols - (undocumented)

Returns:

(undocumented)

Since:

1.3.1

head

```
public Row[] head(int n)
```

Returns the first `n` rows.

Parameters:

n - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

head

```
public Row head()

Returns the first row.

Returns:
    (undocumented)

Since:
    1.3.0
```

first

```
public Row first()

Returns the first row. Alias for head().

Returns:
    (undocumented)

Since:
    1.3.0
```

transform

```
public <U> DataFrame transform(scala.Function1<DataFrame,DataFrame> t)

Concise syntax for chaining custom transformations.

def featurize(ds: DataFrame) = ...

df
  .transform(featurize)
  .transform(...)

Parameters:
    t - (undocumented)

Returns:
    (undocumented)

Since:
    1.6.0
```

map

```
public <R> RDD<R> map(scala.Function1<Row,R> f,
    scala.reflect.ClassTag<R> evidence$4)

Returns a new RDD by applying a function to all rows of this DataFrame.

Parameters:
    f - (undocumented)
    evidence$4 - (undocumented)

Returns:
    (undocumented)

Since:
    1.3.0
```

flatMap

```
public <R> RDD<R> flatMap(scala.Function1<Row,scala.collection.TraversableOnce<R>> f,
    scala.reflect.ClassTag<R> evidence$5)

Returns a new RDD by first applying a function to all rows of this DataFrame, and then flattening the results.

Parameters:
    f - (undocumented)
    evidence$5 - (undocumented)

Returns:
    (undocumented)

Since:
    1.3.0
```

mapPartitions

```
public <R> RDD<R> mapPartitions(scala.Function1<scala.collection.Iterator<Row>,scala.collection.Iterator<R>>> f,
    scala.reflect.ClassTag<R> evidence$6)
```

Returns a new RDD by applying a function to each partition of this `DataFrame`.

Parameters:

`f` - (undocumented)
`evidence$6` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

foreach

```
public void foreach(scala.Function1<Row,scala.runtime.BoxedUnit> f)
```

Applies a function `f` to all rows.

Parameters:

`f` - (undocumented)

Since:

1.3.0

foreachPartition

```
public void foreachPartition(scala.Function1<scala.collection.Iterator<Row>,scala.runtime.BoxedUnit> f)
```

Applies a function `f` to each partition of this `DataFrame`.

Parameters:

`f` - (undocumented)

Since:

1.3.0

take

```
public Row[] take(int n)
```

Returns the first `n` rows in the `DataFrame`.

Running `take` requires moving data into the application's driver process, and doing so with a very large `n` can crash the driver process with `OutOfMemoryError`.

Parameters:

`n` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

takeAsList

```
public java.util.List<Row> takeAsList(int n)
```

Returns the first `n` rows in the `DataFrame` as a list.

Running `take` requires moving data into the application's driver process, and doing so with a very large `n` can crash the driver process with `OutOfMemoryError`.

Parameters:

`n` - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

collect

```
public Row[] collect()
```

Returns an array that contains all of `Rows` in this `DataFrame`.

Running `collect` requires moving all the data into the application's driver process, and doing so on a very large dataset can crash the driver process with `OutOfMemoryError`.

For Java API, use `collectAsList`.

Returns:

(undocumented)

Since:

1.3.0

collectAsList

```
public java.util.List<Row> collectAsList()
```

Returns a Java list that contains all of [Rows](#) in this [DataFrame](#).

Running collect requires moving all the data into the application's driver process, and doing so on a very large dataset can crash the driver process with `OutOfMemoryError`.

Returns:

(undocumented)

Since:

1.3.0

count

```
public long count()
```

Returns the number of rows in the [DataFrame](#).

Returns:

(undocumented)

Since:

1.3.0

repartition

```
public DataFrame repartition(int numPartitions)
```

Returns a new [DataFrame](#) that has exactly `numPartitions` partitions.

Parameters:

`numPartitions` - (undocumented)

Returns:

(undocumented)

Since:

1.3.0

repartition

```
public DataFrame repartition(int numPartitions,  
                             scala.collection.Seq<Column> partitionExprs)
```

Returns a new [DataFrame](#) partitioned by the given partitioning expressions into `numPartitions`. The resulting [DataFrame](#) is hash partitioned.

This is the same operation as "DISTRIBUTE BY" in SQL (Hive QL).

Parameters:

`numPartitions` - (undocumented)

`partitionExprs` - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

repartition

```
public DataFrame repartition(scala.collection.Seq<Column> partitionExprs)
```

Returns a new [DataFrame](#) partitioned by the given partitioning expressions preserving the existing number of partitions. The resulting [DataFrame](#) is hash partitioned.

This is the same operation as "DISTRIBUTE BY" in SQL (Hive QL).

Parameters:

`partitionExprs` - (undocumented)

Returns:

(undocumented)

Since:

1.6.0

coalesce

```
public DataFrame coalesce(int numPartitions)
```

Returns a new [DataFrame](#) that has exactly `numPartitions` partitions. Similar to `coalesce` defined on an RDD, this operation results in a narrow dependency, e.g. if you go from 1000 partitions to 100 partitions, there will not be a shuffle, instead each of the 100 new partitions will claim 10 of the current partitions.

Parameters:

`numPartitions` - (undocumented)

Returns:

(undocumented)

Since:

1.4.0

distinct
<pre>public DataFrame distinct()</pre> Returns a new <code>DataFrame</code> that contains only the unique rows from this <code>DataFrame</code>. This is an alias for <code>dropDuplicates</code>. Returns: (undocumented) Since: 1.3.0

persist
<pre>public DataFrame persist()</pre> Persist this <code>DataFrame</code> with the default storage level (<code>MEMORY_AND_DISK</code>). Returns: (undocumented) Since: 1.3.0

cache
<pre>public DataFrame cache()</pre> Persist this <code>DataFrame</code> with the default storage level (<code>MEMORY_AND_DISK</code>). Returns: (undocumented) Since: 1.3.0

persist
<pre>public DataFrame persist(StorageLevel newLevel)</pre> Persist this <code>DataFrame</code> with the given storage level. Parameters: newLevel - One of: <code>MEMORY_ONLY</code>, <code>MEMORY_AND_DISK</code>, <code>MEMORY_ONLY_SER</code>, <code>MEMORY_AND_DISK_SER</code>, <code>DISK_ONLY</code>, <code>MEMORY_ONLY_2</code>, <code>MEMORY_AND_DISK_2</code>, etc. Returns: (undocumented) Since: 1.3.0

unpersist
<pre>public DataFrame unpersist(boolean blocking)</pre> Mark the <code>DataFrame</code> as non-persistent, and remove all blocks for it from memory and disk. Parameters: blocking - Whether to block until all blocks are deleted. Returns: (undocumented) Since: 1.3.0

unpersist
<pre>public DataFrame unpersist()</pre> Mark the <code>DataFrame</code> as non-persistent, and remove all blocks for it from memory and disk. Returns: (undocumented) Since: 1.3.0

rdd
<pre>public RDD<Row> rdd()</pre> Represents the content of the <code>DataFrame</code> as an RDD of <code>Rows</code>. Note that the RDD is memoized. Once called, it won't change even if you change any query planning related Spark SQL

configurations (e.g. `spark.sql.shuffle.partitions`).

Returns:

(undocumented)

Since:

1.3.0

toJavaRDD

`public JavaRDD<Row> toJavaRDD()`

Returns the content of the `DataFrame` as a `JavaRDD` of `Rows`.

Returns:

(undocumented)

Since:

1.3.0

javaRDD

`public JavaRDD<Row> javaRDD()`

Returns the content of the `DataFrame` as a `JavaRDD` of `Rows`.

Returns:

(undocumented)

Since:

1.3.0

registerTempTable

`public void registerTempTable(java.lang.String tableName)`

Registers this `DataFrame` as a temporary table using the given name. The lifetime of this temporary table is tied to the `SQLContext` that was used to create this `DataFrame`.

Parameters:

`tableName` - (undocumented)

Since:

1.3.0

write

Experimental

`public DataFrameWriter write()`

Interface for saving the content of the `DataFrame` out into external storage.

Returns:

(undocumented)

Since:

1.4.0

toJSON

`public RDD<java.lang.String> toJSON()`

Returns the content of the `DataFrame` as a `RDD` of `JSON` strings.

Returns:

(undocumented)

Since:

1.3.0

inputFiles

`public java.lang.String[] inputFiles()`

Returns a best-effort snapshot of the files that compose this `DataFrame`. This method simply asks each constituent `BaseRelation` for its respective files and takes the union of all results. Depending on the source relations, this may not find all input files. Duplicates are removed.

Returns:

(undocumented)

javaToPython

`protected JavaRDD<byte[]> javaToPython()`

Converts a `JavaRDD` to a `PythonRDD`.

Returns:

(undocumented)

collectToPython

protected int collectToPython()

toSchemaRDDpublic `DataFrame` toSchemaRDD()**Deprecated.** As of 1.3.0, replaced by `toDF()`. This will be removed in Spark 2.0.**Returns:**

(undocumented)

createJDBCTable

```
public void createJDBCTable(java.lang.String url,
                           java.lang.String table,
                           boolean allowExisting)
```

Deprecated. As of 1.340, replaced by `write().jdbc()`. This will be removed in Spark 2.0.Save this `DataFrame` to a JDBC database at `url` under the table name `table`. This will run a `CREATE TABLE` and a bunch of `INSERT INTO` statements. If you pass `true` for `allowExisting`, it will drop any table with the given name; if you pass `false`, it will throw if the table already exists.**Parameters:**

`url` - (undocumented)

`table` - (undocumented)

`allowExisting` - (undocumented)

insertIntoJDBC

```
public void insertIntoJDBC(java.lang.String url,
                          java.lang.String table,
                          boolean overwrite)
```

Deprecated. As of 1.4.0, replaced by `write().jdbc()`. This will be removed in Spark 2.0.Save this `DataFrame` to a JDBC database at `url` under the table name `table`. Assumes the table already exists and has a compatible schema. If you pass `true` for `overwrite`, it will `TRUNCATE` the table before performing the `INSERTS`.The table must already exist on the database. It must have a schema that is compatible with the schema of this RDD; inserting the rows of the RDD in order via the simple statement `INSERT INTO table VALUES (?, ?, ..., ?)` should not fail.**Parameters:**

`url` - (undocumented)

`table` - (undocumented)

`overwrite` - (undocumented)

saveAsParquetFile

public void saveAsParquetFile(java.lang.String path)

Deprecated. As of 1.4.0, replaced by `write().parquet()`. This will be removed in Spark 2.0.Saves the contents of this `DataFrame` as a parquet file, preserving the schema. Files that are written out using this method can be read back in as a `DataFrame` using the `parquetFile` function in `SQLContext`.**Parameters:**`path` - (undocumented)**saveAsTable**

public void saveAsTable(java.lang.String tableName)

Deprecated. As of 1.4.0, replaced by `write().saveAsTable(tableName)`. This will be removed in Spark 2.0.Creates a table from the the contents of this `DataFrame`. It will use the default data source configured by `spark.sql.sources.default`. This will fail if the table already exists.Note that this currently only works with `DataFrames` that are created from a `HiveContext` as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an `insertInto`.When the `DataFrame` is created from a non-partitioned `HadoopFsRelation` with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.**Parameters:**`tableName` - (undocumented)**saveAsTable**

```
public void saveAsTable(java.lang.String tableName,
                       SaveMode mode)
```

Deprecated. As of 1.4.0, replaced by `write().mode(mode).saveAsTable(tableName)`. This will be removed in Spark 2.0.Creates a table from the the contents of this `DataFrame`, using the default data source configured by `spark.sql.sources.default` and `SaveMode.ErrorIfExists` as the save mode.Note that this currently only works with `DataFrames` that are created from a `HiveContext` as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an

RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an insertInto.

When the DataFrame is created from a non-partitioned HadoopFsRelation with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.

Parameters:

- tableName - (undocumented)
- mode - (undocumented)

saveAsTable

```
public void saveAsTable(java.lang.String tableName,
                        java.lang.String source)
```

Deprecated. As of 1.4.0, replaced by write().format(source).saveAsTable(tableName). This will be removed in Spark 2.0.

Creates a table at the given path from the the contents of this DataFrame based on a given data source and a set of options, using SaveMode.ErrorIfExists as the save mode.

Note that this currently only works with DataFrames that are created from a HiveContext as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an insertInto.

When the DataFrame is created from a non-partitioned HadoopFsRelation with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.

Parameters:

- tableName - (undocumented)
- source - (undocumented)

saveAsTable

Experimental

```
public void saveAsTable(java.lang.String tableName,
                        java.lang.String source,
                        SaveMode mode)
```

Deprecated. As of 1.4.0, replaced by write().mode(mode).saveAsTable(tableName). This will be removed in Spark 2.0.

Creates a table at the given path from the the contents of this DataFrame based on a given data source, SaveMode specified by mode, and a set of options.

Note that this currently only works with DataFrames that are created from a HiveContext as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an insertInto.

When the DataFrame is created from a non-partitioned HadoopFsRelation with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.

Parameters:

- tableName - (undocumented)
- source - (undocumented)
- mode - (undocumented)

saveAsTable

```
public void saveAsTable(java.lang.String tableName,
                        java.lang.String source,
                        SaveMode mode,
                        java.util.Map<java.lang.String,java.lang.String> options)
```

Deprecated. As of 1.4.0, replaced by write().format(source).mode(mode).options(options).saveAsTable(tableName). This will be removed in Spark 2.0.

Creates a table at the given path from the the contents of this DataFrame based on a given data source, SaveMode specified by mode, and a set of options.

Note that this currently only works with DataFrames that are created from a HiveContext as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an insertInto.

When the DataFrame is created from a non-partitioned HadoopFsRelation with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.

Parameters:

- tableName - (undocumented)
- source - (undocumented)
- mode - (undocumented)
- options - (undocumented)

saveAsTable

```
public void saveAsTable(java.lang.String tableName,
                        java.lang.String source,
                        SaveMode mode,
                        scala.collection.immutable.Map<java.lang.String,java.lang.String> options)
```

Deprecated. As of 1.4.0, replaced by write().format(source).mode(mode).options(options).saveAsTable(tableName). This will be removed in Spark 2.0.

(Scala-specific) Creates a table from the the contents of this DataFrame based on a given data source, SaveMode specified by mode, and a set of options.

Note that this currently only works with DataFrames that are created from a HiveContext as there is no notion of a persisted catalog in a standard SQL context. Instead you can write an RDD out to a parquet file, and then register that file as a table. This "table" can then be the target of an insertInto.

When the DataFrame is created from a non-partitioned HadoopFsRelation with a single input path, and the data source provider can be mapped to an existing Hive builtin SerDe (i.e. ORC and Parquet), the table is persisted in a Hive compatible format, which means other systems like Hive will be able to read this table. Otherwise, the table is persisted in a Spark SQL specific format.

Parameters:

- tableName - (undocumented)

source - (undocumented)
mode - (undocumented)
options - (undocumented)

save
<pre>public void save(java.lang.String path)</pre> Deprecated. As of 1.4.0, replaced by <code>write().save(path)</code>. This will be removed in Spark 2.0. Saves the contents of this DataFrame to the given path, using the default data source configured by <code>spark.sql.sources.default</code> and <code>SaveMode.ErrorIfExists</code> as the save mode. Parameters: path - (undocumented)

save
<pre>public void save(java.lang.String path, SaveMode mode)</pre> Deprecated. As of 1.4.0, replaced by <code>write().mode(mode).save(path)</code>. This will be removed in Spark 2.0. Saves the contents of this DataFrame to the given path and <code>SaveMode</code> specified by mode, using the default data source configured by <code>spark.sql.sources.default</code>. Parameters: path - (undocumented) mode - (undocumented)

save
<pre>public void save(java.lang.String path, java.lang.String source)</pre> Deprecated. As of 1.4.0, replaced by <code>write().format(source).save(path)</code>. This will be removed in Spark 2.0. Saves the contents of this DataFrame to the given path based on the given data source, using <code>SaveMode.ErrorIfExists</code> as the save mode. Parameters: path - (undocumented) source - (undocumented)

save
<pre>public void save(java.lang.String path, java.lang.String source, SaveMode mode)</pre> Deprecated. As of 1.4.0, replaced by <code>write().format(source).mode(mode).save(path)</code>. This will be removed in Spark 2.0. Saves the contents of this DataFrame to the given path based on the given data source and <code>SaveMode</code> specified by mode. Parameters: path - (undocumented) source - (undocumented) mode - (undocumented)

save
<pre>public void save(java.lang.String source, SaveMode mode, java.util.Map<java.lang.String, java.lang.String> options)</pre> Deprecated. As of 1.4.0, replaced by <code>write().format(source).mode(mode).options(options).save(path)</code>. This will be removed in Spark 2.0. Saves the contents of this DataFrame based on the given data source, <code>SaveMode</code> specified by mode, and a set of options. Parameters: source - (undocumented) mode - (undocumented) options - (undocumented)

save
<pre>public void save(java.lang.String source, SaveMode mode, scala.collection.immutable.Map<java.lang.String, java.lang.String> options)</pre> Deprecated. As of 1.4.0, replaced by <code>write().format(source).mode(mode).options(options).save(path)</code>. This will be removed in Spark 2.0. (Scala-specific) Saves the contents of this DataFrame based on the given data source, <code>SaveMode</code> specified by mode, and a set of options Parameters: source - (undocumented) mode - (undocumented) options - (undocumented)

insertInto

```
public void insertInto(java.lang.String tableName,
    boolean overwrite)
```

Deprecated. As of 1.4.0, replaced by `write().mode(SaveMode.Append|SaveMode.Overwrite).saveAsTable(tableName)`. This will be removed in Spark 2.0.
Adds the rows from this RDD to the specified table, optionally overwriting the existing data.

Parameters:

- tableName - (undocumented)
- overwrite - (undocumented)

insertInto

```
public void insertInto(java.lang.String tableName)
```

Deprecated. As of 1.4.0, replaced by `write().mode(SaveMode.Append).saveAsTable(tableName)`. This will be removed in Spark 2.0.
Adds the rows from this RDD to the specified table. Throws an exception if the table already exists.

Parameters:

- tableName - (undocumented)